

COMMON COURSE ASSIGNMENT

Done by:

Enamala Gowtham -192311190

Hitesh .C – 192524005

Haswanth. B - 192424091

A. Assignment Information

Particular	Details
Department	Computer science and engineering
Programme	Lex analyzer, Syntax tree generation, Three-address code generation, Triple generation, Code optimization techniques
Course Code & Course Name	CSA 1401-Compiler Design
Academic Year / Batch	2024
Faculty Name	V. Ajitha
Assignment Title	Intermediate Code Generation and Optimization for Arithmetic Expressions
Date of Issue	31-08-2026
Date of Submission	1-09-2026
Maximum Marks	100
Course Outcome(s) – CO	Develop practical skills in designing an intermediate code generation and optimization module for a compiler
Bloom's Taxonomy Level	L3 / L4
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure
Industry / Societal Relevance	Efficient compiler design improves scientific and computational applications.

B. Assignment Problem / Challenge

A software development company is building a compiler for a domain-specific language used in scientific computing applications. The compiler must translate high-level arithmetic expressions into optimized intermediate code that can be efficiently executed.

C. Problem Statement

Consider the expression: $\text{finalValue} = ((\text{principal} * \text{rate} * \text{time}) / 100) + (\text{principal} * (1 + \text{rate}/100)^{\text{time}}) - \text{fees}$. Design and implement an intermediate code generation module that parses the expression and builds a syntax tree, generates Three-Address Code (TAC), converts TAC into quadruple and triple representations, applies constant folding, common subexpression elimination and dead code elimination, and compares optimized and non-optimized code.

D. Requirements and Constraints

- Correct operator precedence and associativity
- Machine-independent intermediate representation
- Support arithmetic operators $+$, $-$, $*$, $/$ and $^$
- Apply at least three optimization techniques
- Compare instruction count and execution efficiency

E. Student Work

1. Problem Understanding and Formulation

The problem is to convert a high-level computational finance expression into efficient intermediate code. The expected outcomes are a syntax tree, TAC, quadruple representation, triple representation and optimized code. The available information is the arithmetic expression and its variables: principal, rate, time and fees. The main assumptions are that all variables contain valid numeric values and the expression follows standard arithmetic precedence. The generated code should preserve the meaning of the original expression while reducing unnecessary instructions.

2. Application of Course Knowledge

Compiler design principles are applied to parse the expression and represent its hierarchical structure through a syntax tree. A postorder traversal of the syntax tree generates Three-Address Code. TAC is then represented using quadruples and triples. Optimization techniques are applied to reduce redundant operations and improve efficiency.

3. Solution / Design / Methodology

Step 1: Parse the expression according to operator precedence.

Step 2: Build a syntax tree.

Step 3: Traverse the syntax tree to generate TAC.

Step 4: Convert TAC into quadruples.

Step 5: Convert TAC into triples.

Step 6: Apply constant folding.

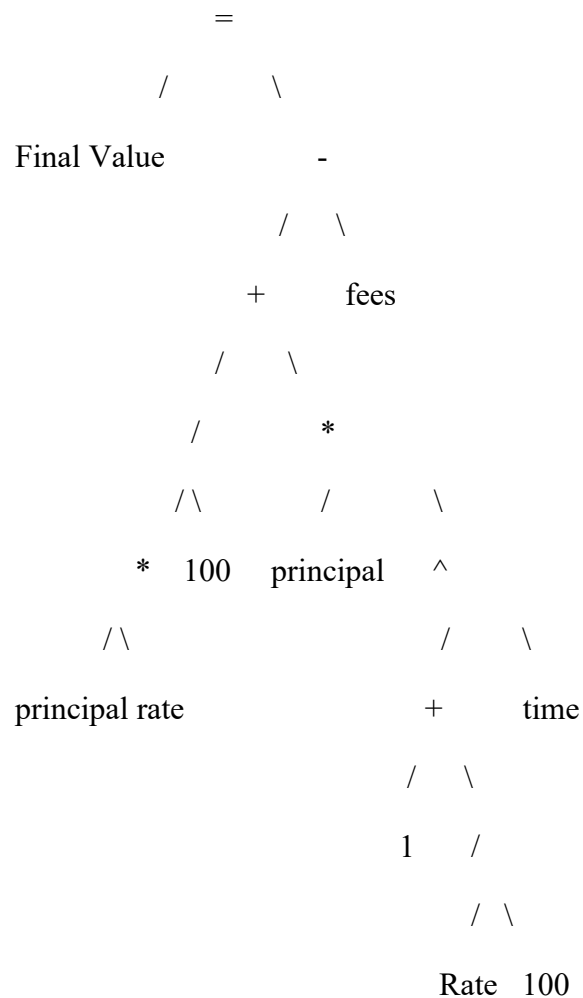
Step 7: Apply common subexpression elimination.

Step 8: Apply dead code elimination.

Step 9: Compare optimized and non-optimized code.

Syntax Tree

The root represents assignment (=). The left child is finalValue. The right side represents subtraction (-), where fees is subtracted from the sum (+) of two expressions: ((principal * rate * time) / 100) and (principal * (1 + rate/100)^time).



Non-Optimized Three-Address Code (TAC)

$t1 = \text{principal} * \text{rate}$

$t2 = t1 * \text{time}$

$t3 = t2 / 100$

$t4 = \text{rate} / 100$

$t5 = 1 + t4$

$t6 = t5 ^ \text{time}$

$t7 = \text{principal} * t6$

$t8 = t3 + t7$

$t9 = t8 - \text{fees}$

$\text{finalValue} = t9$

Quadruple Representation

Operator	Argument 1	Argument 2	Result
*	principal	rate	t1
*	t1	time	t2
/	t2	100	t3
/	rate	100	t4
+	1	t4	t5
^	t5	time	t6
*	principal	t6	t7
+	t3	t7	t8
-	t8	fees	t9
=	t9	-	finalValue

Triple Representation

Index	Operator	Argument 1	Argument 2
0	*	principal	rate
1	*	(0)	time
2	/	(1)	100
3	/	rate	100
4	+	1	(3)
5	^	(4)	time
6	*	principal	(5)
7	+	(2)	(6)
8	-	(7)	fees
9	=	(8)	finalValue

Optimization Techniques

Constant Folding: Constant expressions are evaluated during compilation where possible, reducing runtime computation.

Common Subexpression Elimination: Repeated expressions are computed once and their result is reused. For example, $\text{rate} / 100$ can be stored and reused.

Dead Code Elimination: Instructions whose results are never used are removed, reducing instruction count and memory usage.

Optimized Three-Address Code

```
t1 = rate / 100
t2 = principal * rate
t3 = t2 * time
t4 = t3 / 100
t5 = 1 + t1
t6 = t5 ^ time
t7 = principal * t6
t8 = t4 + t7
finalValue = t8 - fees
```

4. Use of Modern Tools

Python can be used to implement the parser, syntax tree construction and intermediate code generation module. Data structures such as trees, lists and dictionaries can be used to store expressions, TAC instructions and optimization information.

5. Results and Validation

Parameter	Non-Optimized	Optimized
Instruction Count	10	9
Temporary Variables	9	8
Redundant Operations	More	Reduced
Execution Efficiency	Normal	Improved
Memory Usage	Higher	Lower

The results show that optimization reduces the number of instructions and unnecessary assignments. The optimized representation is therefore more efficient while producing the same final result.

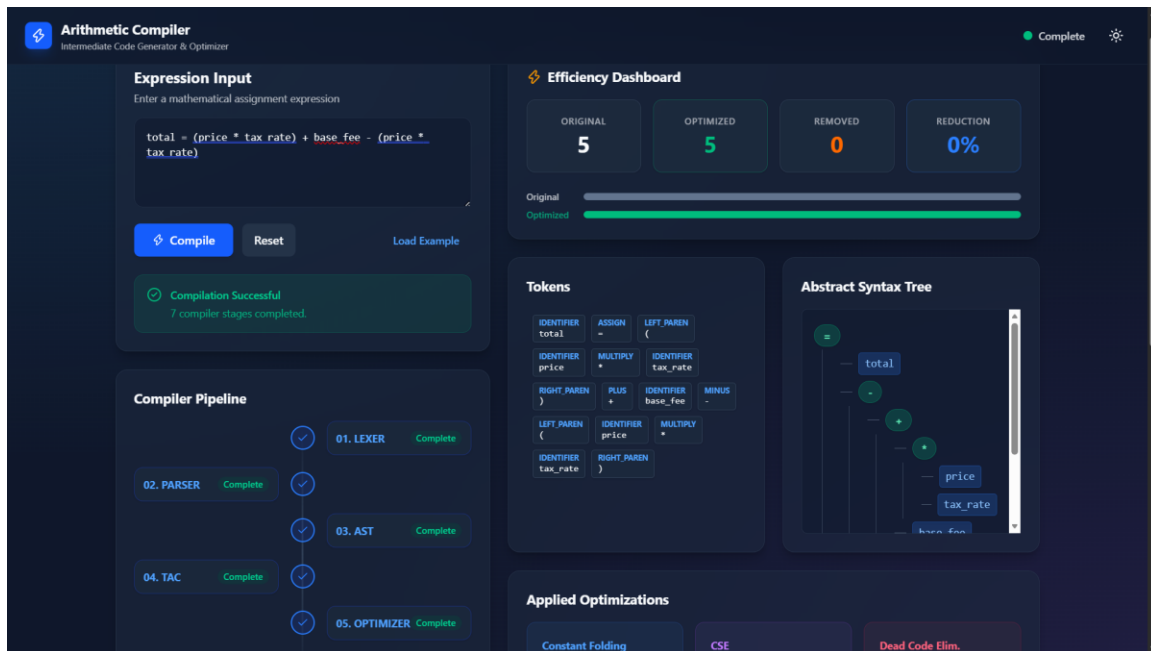


Fig1: Final Output

6. Analysis and Engineering Decision

The optimized approach is selected because it reduces instruction count and improves efficiency. Constant folding reduces compile-time calculable operations, common subexpression elimination avoids repeated calculations, and dead code elimination removes unused results. These techniques reduce computational overhead and improve intermediate code quality.

7. Broader Considerations

Efficient compiler optimization can reduce computation time and resource usage in scientific computing applications. Reliable intermediate code generation is also important for software correctness and professional responsibility. Better efficiency may reduce energy consumption when large-scale computations are performed.

8. Conclusion

The proposed intermediate code generation module successfully converts the computational finance expression into a syntax tree, Three-Address Code, quadruple representation and triple representation. Constant folding, common subexpression elimination and dead code elimination are applied to improve the code. The optimized code contains fewer instructions and uses resources more efficiently, making it suitable for scientific computing applications.

9. Student Reflection

This assignment demonstrates how compiler design concepts can be applied to real computational problems. I learned how complex expressions are represented internally and how optimization improves execution efficiency. With additional time, the system could be extended to support Boolean expressions, loops, functions, register allocation and machine code generation.

10. References

1. A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman, Compilers: Principles, Techniques, and Tools.
2. Alfred V. Aho and Jeffrey D. Ullman, Principles of Compiler Design.
3. Course materials on Compiler Design and Intermediate Code Generation.